

Garvix Ops SRS

Software Requirements Specification

Garvix Ops — Workforce Management & Attendance System

Field	Detail
Document Version	1.0.0
Status	Final
Prepared For	Shorthills AI / Internal Use
Application	Garvix Ops
Deployment	Web (SPA) — GitHub Pages / Render
Date	May 2026

Table of Contents

- 1. [Introduction](#)
- 2. [System Overview](#)
- 3. [User Roles & Access Control](#)
- 4. [Functional Requirements](#)
 - 4.1 Authentication
 - 4.2 Attendance Management
 - 4.3 Employee Management
 - 4.4 Task Management
 - 4.5 Meeting Management
 - 4.6 Campaigns
 - 4.7 Team Target Tracking
 - 4.8 Personal Productivity (To-Do & Notes)

- 4.9 Credentials Vault
- 4.10 Daily Motivation Quotes
- 4.11 US Time Zone Clock
- 4.12 MOM Generator Integration
- 4.13 Profile & Photo Setup
- 4.14 Theme & UI Settings

5. Non-Functional Requirements

6. Technology Stack

7. System Architecture

8. Data Models (Firestore Collections)

9. Security Design

10. UI/UX Design Principles

11. External Integrations

12. Constraints & Assumptions

13. Glossary

1. Introduction

1.1 Purpose

This document specifies the full software requirements for **Garvix Ops**, a web-based workforce management platform built for Shorthills AI. It covers functional requirements, non-functional requirements, system architecture, data models, and the complete technology stack.

1.2 Scope

Garvix Ops provides two distinct portals — an **Admin (Manager) Portal** and an **Employee Portal** — served from a single React SPA. The platform handles:

- Real-time biometric-grade attendance marking (face recognition + GPS geofencing)
- Employee lifecycle management
- Task assignment and tracking
- Meeting target management and reporting
- Campaign assignment
- Personal productivity tools (to-do lists, notes, credentials)
- Live US timezone clocks and daily motivation quotes

1.3 Intended Audience

Audience	Purpose
Development Team	Implementation reference
QA Team	Test case derivation
Admin / Managers	Feature understanding and operations guide
Employees	Portal feature reference

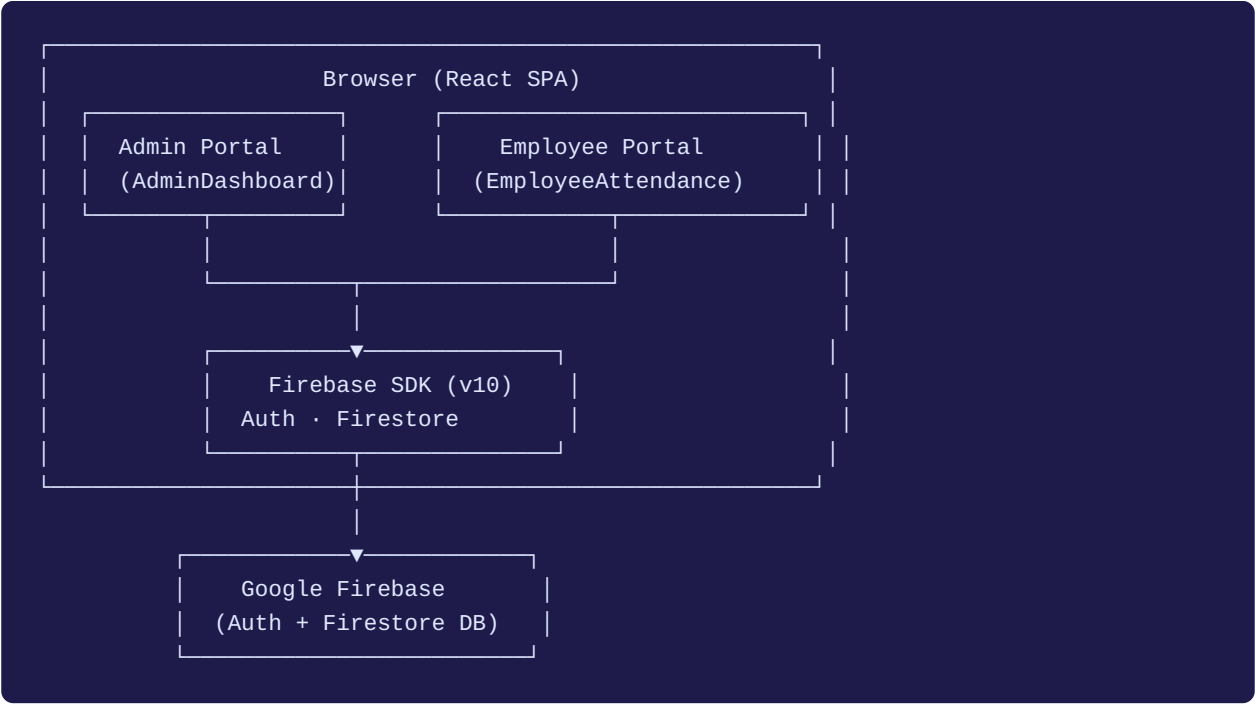
1.4 Definitions

Term	Definition
IST	Indian Standard Time (UTC+5:30) — all server timestamps use IST
MOM	Minutes of Meeting
SPA	Single Page Application
Face Match	Euclidean distance comparison of 128-point face descriptors
Geofence	700-metre radius around the registered office location

2. System Overview

Garvix Ops is a **real-time, role-based workforce management system** that enables managers to monitor and manage their team while giving employees a seamless interface for daily work operations.

2.1 High-Level Architecture



2.2 Deployment

- **Hosting:** GitHub Pages (static build) / Render
- **Build Tool:** Vite (production bundle)
- **CI/CD:** Manual `npm run build` → deploy

3. User Roles & Access Control

3.1 Roles

Role	Login Page	Access Scope
Admin	<code>/admin/login</code>	Full system access — all employees, reports, settings
Employee	<code>/employee/login</code>	Personal data only — own attendance, tasks, meetings

3.2 Role Separation Enforcement

- Firebase Authentication provides unique UIDs per user.
- The admin uses a dedicated Firebase Auth account separate from all employee accounts.

- A **secondary Firebase app instance** (`secondaryAuth`) is used to create employee accounts without signing out the currently logged-in admin.
 - Employee portal data is always scoped to the authenticated user's UID (`employeeUid`).
 - Employees cannot view other employees' attendance, task details, or personal data.
-

4. Functional Requirements

4.1 Authentication

4.1.1 Admin Login

- Email/password authentication via Firebase Auth.
- Session persists using `browserSessionPersistence` (clears on tab close).
- Protected route `/admin/*` — redirects to login if unauthenticated.

4.1.2 Employee Login

- Email/password authentication via Firebase Auth.
- Employees are provisioned by the admin; they cannot self-register.
- On first login, employees complete a one-time **setup wizard** (name, phone, profile photo).

4.1.3 Session Management

- Stale `firebase:*` keys are purged from `localStorage` on each app load to prevent session conflicts.
 - Secondary app (`inMemoryPersistence`) ensures admin session is not affected when creating employees.
-

4.2 Attendance Management

Attendance is the core feature of the platform, designed for fraud-resistant marking via two optional security layers.

4.2.1 Attendance Methods

Method	Description
Simple	One-click sign-in/sign-out; no biometric or location check
Face Match	Webcam captures frame → 128-point descriptor compared to stored photo
GPS Check	Device GPS compared to office geofence (700m radius, Gurugram)
Face + GPS	Both checks must pass before attendance is recorded

The active method is configured by the admin in **Profile Setup Settings** and stored in the `appSettings` Firestore document.

4.2.2 Face Recognition

- Powered by `@vladmandic/face-api` (TensorFlow.js-based).
- Three neural network models loaded from jsDelivr CDN on first use (cached by browser):
 - `ssdMobilenetv1` — face detection
 - `faceLandmark68Net` — 68-point facial landmark detection
 - `faceRecognitionNet` — 128-point face descriptor extraction
- Face comparison uses **Euclidean distance** with threshold `0.65` — distances below this are a match.
- The stored face descriptor (128-element float array) is saved to the employee's Firestore document.

4.2.3 GPS Geofencing

- Office location: **Ambience Mall, Sector 24, Gurugram** (lat: 28.5027, lng: 77.0929).
- Allowed radius: **700 metres**.
- Distance calculated using the **Haversine formula** for accurate great-circle distance.
- GPS request timeout: 15 seconds; maximum cached age: 30 seconds; high-accuracy mode enabled.

4.2.4 Attendance Record Structure

Each sign-in/sign-out creates a record in the `attendance` Firestore collection:

```
{
  employeeUid: string, // Firebase Auth UID
  employeeId: string, // Company employee code
  employeeName: string,
  date: string, // "YYYY-MM-DD" in IST
  month: string, // "YYYY-MM" in IST
  type: "sign-in" | "sign-out",
  timestamp: Firestore ServerTimestamp,
  method: "simple" | "face" | "gps" | "face+gps",
  faceMatched: boolean (optional),
  withinOffice: boolean (optional),
  distance: number (optional, metres)
}
```

4.2.5 Attendance Popup

- The attendance popup is triggered **only** when the employee manually clicks “Sign In” or “Sign Out”.
- No automatic popups; no recurring intervals.
- Popup displays the applicable verification step (face scan / GPS check / both / simple button).

4.2.6 Working Hours

- Total working hours are displayed only after sign-out.
- Computed as the difference between sign-in timestamp and sign-out timestamp.

4.2.7 Attendance Views

- **Employee:** Sees own today’s record, recent attendance history.
- **Admin:** Sees all employees’ sign-in/sign-out records in parallel columns; can filter by date and employee.

4.3 Employee Management

4.3.1 Add Employee

- Admin navigates to `/admin/add-employee`.
- Form collects: Full Name, Employee ID (company code), Email, Phone, Role/Designation.
- Firebase Auth account is created using the **secondary app** (does not sign out admin).
- Employee doc is written to the `employees` Firestore collection with the new UID as the document ID.

4.3.2 Edit Employee

- Admin can edit any employee's details via an inline modal.
- Editable fields: Name, Phone, Designation, Employee ID.

4.3.3 Employee List View

- Admin Employees page: 4-column grid, compact cards, all employees visible without scrolling.
- Each card shows: avatar/initials, name, employee ID, designation, email, active campaign chips.
- Search/filter bar filters by name, email, or employee ID.

4.3.4 Employee Profile Photo

- Admin can enable/disable profile photo requirement via **Profile Setup Settings**.
 - When toggled on, employees receive a real-time in-app notification to upload a profile photo.
 - Photo is taken via webcam and stored as a base64 data URL in the employee's Firestore doc.
 - Photo is also processed into a face descriptor and stored for future face-match attendance.
 - Saving the photo is mandatory before the employee can mark attendance when face-match is enabled.
-

4.4 Task Management

4.4.1 Assign Task (Admin)

- Admin selects an employee and creates a task with:
 - Title, Description, Priority (High / Medium / Low), Due Date.
- Tasks are stored in the `tasks` Firestore collection, linked by employee UID.

4.4.2 Employee Task View

- Employees see their assigned tasks in the **Assigned Tasks** page.
- Tasks display priority colour coding, due date, and completion status.
- Unread task badge shown on the sidebar.

4.4.3 Task Notifications

- New task assignments trigger a real-time notification indicator in the employee portal.
 - Admin is notified of task counts on the dashboard.
-

4.5 Meeting Management

4.5.1 Assign Meeting Target (Admin)

- Admin assigns each employee a monthly meeting target (scheduled + completed meetings).
- Stored per employee per month in the `meetings` Firestore collection.

4.5.2 Meeting Report (Admin)

- Accessible from the Admin sidebar under Team.
- Displays aggregated team meeting statistics with:
 - **KPI cards** (total target, total completed, total scheduled, overall achievement %).
 - **RadialGauge** (SVG arc chart) for team completion percentage.
 - **Per-employee table** with month-selectable multi-filter.
 - **6-month history table**.
- All font sizes are tuned to prevent overlap; SVG text uses proportional sizing.

4.5.3 Meeting Report (Employee)

- Employees see only their own meeting data.
 - Same RadialGauge and KPI card layout, scoped to the current user's records.
-

4.6 Campaigns

4.6.1 Create Campaign

- Admin creates campaigns by entering a campaign name.
- Saved to the `campaigns` Firestore collection with timestamp.

4.6.2 Assign Campaign

- Admin selects a campaign and assigns it to one or more employees.
- Each employee can hold **multiple active campaigns** simultaneously (stored as an array).
- Data model supports both legacy single-campaign (`campaign: {}`) and new multi-campaign (`campaigns: []`) formats for backwards compatibility.
- Assignment UI shows each employee's current campaigns as removable chips.

4.6.3 Employee Campaign View

- Employee dashboard displays **all assigned campaigns** as labelled chips in a dedicated widget.

- Widget title shows the count: “Assigned Campaigns (N)”.
 - Falls back gracefully to legacy single-campaign field if the array is absent.
-

4.7 Team Target Tracking

4.7.1 Set Monthly Target

- Admin sets a numeric monthly meeting target via the **Team Target** page.
- Target stored in `teamTargets` Firestore collection keyed by `{uid}_{YYYY-MM}`.

4.7.2 Progress Visualisation

- **Radial ring SVG chart** shows completion percentage for the selected month.
- Stats grid: Target, Completed, Scheduled, Remaining.
- Progress ring uses a purple-to-blue gradient with drop-shadow glow.

4.7.3 6-Month History Table

- Displays Target, Completed, Scheduled, Achievement % for the last 6 months.
 - Current month highlighted in purple with a “Current” badge.
 - Completion shown as both a mini progress bar and a percentage label.
-

4.8 Personal Productivity (To-Do & Notes)

4.8.1 My To-Do

- Both admin and employee portals include a personal To-Do list.
- Tasks are categorised as: **Overdue**, **Today**, **Tomorrow**, and future items.
- Priority colour coding: High = red, Medium = amber, Low = blue/green.
- **Dashboard widget:** To-Do list and US Clock shown side-by-side in a split panel.
 - Left half: grouped todo items (scrollable, max height 220px for employee / 340px for admin).
 - Right half: live US time zone clock (see §4.11).

4.8.2 My Notes

- Freeform notes per user stored in Firestore.
 - Rich text-style formatting with timestamps.
-

4.9 Credentials Vault

- Both portals include a **My Credentials** page.
 - Employees and admin can securely store login credentials (service name, username, password).
 - Data scoped to the individual user's Firestore sub-collection.
 - Passwords are masked by default with a show/hide toggle.
-

4.10 Daily Motivation Quotes

- A curated set of **30 sales motivation quotes** cycles through the year.
 - Quote selection: `quotes[dayOfYear % 30]` — computed in IST timezone.
 - The same quote is shown all day and rotates exactly at IST midnight.
 - Displayed in both portals:
 - **Admin dashboard:** Below live date/time in the greeting hero widget.
 - **Employee dashboard:** Below live date/time in the frosted hero banner.
 - Quote is styled in italic with a 💡 prefix in a semi-transparent frosted card.
-

4.11 US Time Zone Clock

A live clock showing the current time across four US time zones, updating every second.

Label	Time Zone	Colour
ET	America/New_York	Purple
CT	America/Chicago	Blue
MT	America/Denver	Green
PT	America/Los_Angeles	Amber

- Displayed in **both** admin and employee dashboards as the right half of the To-Do split panel.
 - Uses `Intl.DateTimeFormat` with `hour12: true`, showing HH:MM:SS.
 - Clock ticks via a `setInterval` running every 1 second.
 - Uses the existing `clock` state (admin) or `currentTime` state (employee), each updated every 1 second.
-

4.12 MOM Generator Integration

- A **Generate MOM** link is present in the sidebar of both portals.
 - Clicking opens `https://mom-generator-jb6a.onrender.com/` in a new browser tab (`noopener, noreferrer`).
 - Positioned under the **Team** group (admin) and **Work** group (employee).
 - Navigation within the app is unaffected — no internal page state change occurs.
-

4.13 Profile & Photo Setup

4.13.1 Admin Controls (Profile Setup Settings)

- Admin toggles:
 - **Require Profile Photo** — triggers employee notification and mandates photo before attendance.
 - **Attendance Method** — choose Simple / Face Match / GPS / Face+GPS.

4.13.2 Employee Photo Upload

- When profile photo is required, the employee sees a persistent banner notification.
 - Clicking the notification opens the **Profile Modal** with a webcam capture interface.
 - `WebcamCapture` component requests camera permission and captures a snapshot.
 - Photo is processed client-side: both the base64 image and the 128-point face descriptor are saved.
 - Once saved, the attendance marking flow can proceed with face-match verification.
-

4.14 Theme & UI Settings

- **Dark / Light mode** toggle available in the top navigation bar.
 - Theme state managed via `ThemeContext` with CSS custom properties.
 - Dark mode colour palette:
 - `--bg` : `#0f0f14` (deep near-black)
 - `--surface` : `#16161f`
 - `--surface-s` : `#1e1e2a`
 - `--border` : `rgba(255, 255, 255, 0.08)`
 - `--text` : `#f0f0f8`, `--text-2` : `#a0a0b8`, `--text-3` : `#606078`
-

5. Non-Functional Requirements

5.1 Performance

Metric	Target
Initial page load	< 3 seconds on 4G connection
Firestore real-time latency	< 500ms for snapshot updates
Face model load (first use)	< 8 seconds (CDN, then cached)
Subsequent face detection	< 2 seconds per frame
Clock tick accuracy	±50ms (browser <code>setInterval</code>)

5.2 Reliability

- All Firestore listeners use `onSnapshot` for real-time sync; no manual polling.
- Firebase client SDK handles reconnection automatically on network loss.
- App gracefully degrades: if face models fail to load, attendance can fall back to simple mode.

5.3 Security

- All Firebase credentials are stored as Vite environment variables (`VITE_FIREBASE_*`), never hardcoded.
- Employee records are accessible only by their own UID and the admin.
- Session ends on tab/browser close (`browserSessionPersistence`).
- External links open with `noopener, noreferrer` to prevent tab-napping.
- No sensitive data (face descriptors, credentials) is logged to the console in production.

5.4 Scalability

- Firestore's serverless architecture scales automatically with team size.
- `memoryLocalCache` mode used for Firestore — avoids IndexedDB conflicts in multi-tab scenarios.
- Client-side sorting used throughout (never `orderBy()` + `where()` combined) to avoid requiring composite indexes.

5.5 Usability

- Fully responsive — desktop (1024px+), tablet (768px), and mobile (375px+).

- Collapsible sidebar on mobile with hamburger toggle.
- Consistent dark-themed design language across both portals.
- All timestamps displayed in IST using `Intl.DateTimeFormat`.

5.6 Availability

- Hosted as a static SPA — no server-side downtime dependency (GitHub Pages / CDN).
- Firebase SLA: 99.95% uptime for Auth and Firestore.

6. Technology Stack

6.1 Frontend

Technology	Version	Role
React	18.3.1	UI component framework
React DOM	18.3.1	DOM rendering
React Router DOM	6.23.1	Client-side routing (<code>BrowserRouter</code>)
Vite	5.2.13	Build tool, dev server, HMR
Tailwind CSS	3.4.4	Utility-first CSS framework
PostCSS	8.4.38	CSS processing pipeline
Autoprefixer	10.4.19	Vendor prefix automation

6.2 AI / Biometrics

Technology	Version	Role
@vladmandic/face-api	1.7.15	Face detection, landmark extraction, recognition
TensorFlow.js (bundled)	—	Underlying ML inference engine
SSD MobileNet V1	—	Real-time face detection model
Face Landmark 68-Net	—	68-point facial landmark model
Face Recognition Net	—	128-point descriptor embedding model
Euclidean Distance	—	Face similarity metric (threshold: 0.65)
Web Geolocation API	—	GPS coordinate retrieval (browser native)
Haversine Formula	—	Great-circle GPS distance calculation

6.3 Backend / Infrastructure

Technology	Version	Role
Firebase JS SDK	10.12.0	Auth + Firestore client
Firebase Auth	—	User authentication, session management
Cloud Firestore	—	Real-time NoSQL database
Firebase App (multi)	—	Secondary app instance for admin-side account creation

6.4 Developer Tools

Technology	Version	Role
ESLint	8.57.0	Static code analysis
eslint-plugin-react	7.34.2	React-specific lint rules
eslint-plugin-react-hooks	4.6.2	Hooks lint rules

6.5 External Services

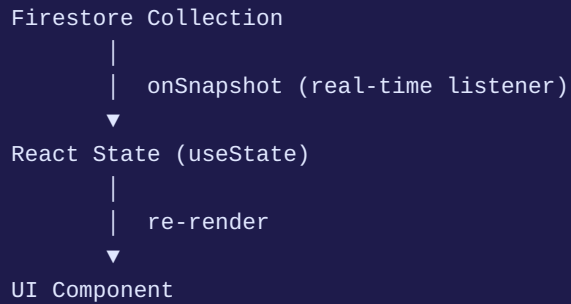
Service	Purpose
jsDelivr CDN	Face recognition model hosting
MOM Generator (Render)	Minutes of Meeting generation tool
GitHub Pages / Render	Application hosting

7. System Architecture

7.1 Component Tree (Simplified)

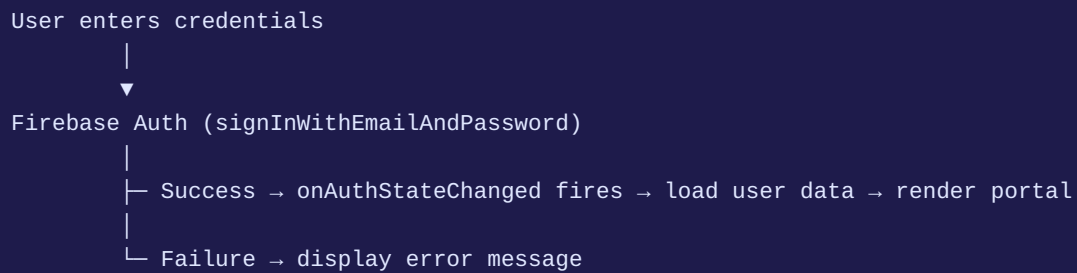
```
App (BrowserRouter)
├─ /admin/login      → AdminLogin
├─ /admin/add-employee → AddEmployee
├─ /admin/*          → AdminDashboard
│   └─ Sidebar (nav groups: Main, Team, Personal, Settings)
│   └─ Navbar
│   └─ Page Content (conditional rendering by `page` state)
│       ├── DashboardPage (inline)
│       ├── EmployeesPage
│       ├── AssignTaskPage
│       ├── AssignMeetingTarget
│       ├── TeamTargetPage
│       ├── MeetingReport
│       ├── CampaignsPage
│       ├── TodoPage
│       ├── NotesPage
│       ├── CredentialsPage
│       └─ ProfileSetupSettings
├─ /employee/login   → EmployeeLogin
├─ /employee/setup   → EmployeeSetup
└─ /employee/*       → EmployeeAttendance
    ├── Sidebar (nav groups: Overview, Work, Personal)
    ├── Navbar
    ├── AttendancePopup (modal, shown on manual trigger only)
    └─ Page Content
        ├── DashboardPage (inline)
        ├── MyAttendancePage
        ├── AssignedTasksPage
        ├── AssignedMeetings
        ├── MeetingReportEmployee
        ├── TodoPage
        ├── NotesPage
        └─ CredentialsPage
```

7.2 Real-Time Data Flow



All data in the app flows from Firestore via `onSnapshot` listeners, which push updates to React state and trigger automatic UI re-renders. There is no REST API polling layer.

7.3 Authentication Flow



8. Data Models (Firestore Collections)

8.1 `employees` Collection

Document ID: Firebase Auth UID

```

{
  uid:          string,           // Firebase Auth UID (same as doc ID)
  employeeId:   string,           // Company employee code
  name:         string,
  email:        string,
  phone:        string,
  designation:  string,
  photoURL:     string,           // base64 dataURL
  faceDescriptor: number[],       // 128-point float array
  campaigns:    [{ id, name }],   // Array of assigned campaigns
  campaign:     { id, name, assignedAt } | null, // Legacy (deprecated)
  createdAt:    Timestamp
}

```

8.2 attendance Collection

```

{
  employeeUid:  string,
  employeeId:   string,
  employeeName: string,
  date:         string,           // "YYYY-MM-DD" IST
  month:        string,           // "YYYY-MM" IST
  type:         "sign-in" | "sign-out",
  timestamp:    Timestamp,
  method:       "simple" | "face" | "gps" | "face+gps",
  faceMatched:  boolean,
  withinOffice: boolean,
  distance:     number            // metres from office
}

```

8.3 tasks Collection

```

{
  employeeUid:  string,
  title:        string,
  description:   string,
  priority:     "high" | "medium" | "low",
  dueDate:      string,
  completed:    boolean,
  createdAt:    Timestamp,
  assignedBy:   string            // admin UID
}

```

8.4 `meetings` Collection

```
{
  employeeUid: string,
  employeeName: string,
  month: string, // "YYYY-MM"
  scheduled: number,
  completed: number,
  updatedAt: Timestamp
}
```

8.5 `teamTargets` Collection

Document ID: `{adminUid}_{YYYY-MM}`

```
{
  target: number,
  month: string,
  updatedAt: Timestamp
}
```

8.6 `campaigns` Collection

```
{
  name: string,
  createdAt: Timestamp,
  createdBy: string // admin UID
}
```

8.7 `appSettings` Collection

Document ID: `main`

```
{
  requireProfilePhoto: boolean,
  attendanceMethod: "simple" | "face" | "gps" | "face+gps",
  updatedAt: Timestamp
}
```

8.8 `todos` Collection

```
{
  userId:    string,
  title:     string,
  priority:  "high" | "medium" | "low",
  date:      string,           // due date "YYYY-MM-DD"
  completed: boolean,
  createdAt: Timestamp
}
```

8.9 `notes` Collection

```
{
  userId:    string,
  content:   string,
  updatedAt: Timestamp
}
```

8.10 `credentials` Collection

```
{
  userId:    string,
  service:   string,
  username:  string,
  password:  string,           // stored as plain text in Firestore (user-managed)
  createdAt: Timestamp
}
```

9. Security Design

9.1 Authentication

- Firebase Auth with `browserSessionPersistence` — sessions expire on tab close.
- No JWT management required client-side; Firebase SDK handles token refresh automatically.
- Employees cannot register themselves; all accounts are provisioned by admin.

9.2 Environment Variables

All Firebase configuration values are injected at build time via Vite environment variables:

```
VITE_FIREBASE_API_KEY
VITE_FIREBASE_AUTH_DOMAIN
VITE_FIREBASE_PROJECT_ID
VITE_FIREBASE_STORAGE_BUCKET
VITE_FIREBASE_MESSAGING_SENDER_ID
VITE_FIREBASE_APP_ID
```

These are set in the hosting environment (Render) and are never committed to source control.

9.3 Data Access Patterns

- Employee data queries are always scoped with `where('employeeUid', '==', user.uid)`.
- Admin accesses all employees' data via broad collection reads.
- Firestore Security Rules (server-side) should be configured to enforce role-based access at the database level.

9.4 External Link Safety

All external links (e.g., MOM Generator) are opened with:

```
window.open(url, '_blank', 'noopener,noreferrer')
```

This prevents the opened page from accessing the app's `window.opener` reference.

10. UI/UX Design Principles

10.1 Design Language

- **Dark-first** — deep near-black backgrounds with subtle purple/blue accent gradients.
- **Glass morphism** cards — `rgba` backgrounds with subtle borders and box shadows.
- **Purple-to-blue gradient** is the primary brand accent: `linear-gradient(135deg, #7c3aed, #3b82f6)`.
- All interactive elements have hover state transitions.

10.2 Typography

- Base body: 12–14px, system font stack.
- Section headers: `text-xs`, `font-semibold`, `uppercase`, `tracking-wider`.
- KPI numbers: 18–22px, `font-bold`.

- Sidebar labels: 12px.
- Timestamps and metadata: 10–11px.

10.3 Responsive Layout

- **Desktop (≥1024px):** Fixed sidebar (256px), main content fills remaining width.
- **Mobile (<768px):** Sidebar hidden by default, toggled via hamburger; slides in as overlay.
- Dashboard widgets use CSS Grid with responsive column counts.

10.4 Real-Time Indicators

- Live clock updates every second for both IST and US time zones.
- Animated pulse dots on sidebar items with pending items.
- Smooth SVG progress ring transitions using CSS `transition: stroke-dasharray 1s ease`.

11. External Integrations

11.1 MOM Generator

Property	Value
URL	<code>https://mom-generator-jb6a.onrender.com/</code>
Type	External web application
Integration	Browser redirect (new tab)
Available In	Admin Portal (Team group) + Employee Portal (Work group)

11.2 jsDelivr CDN (Face Models)

Property	Value
URL	<code>https://cdn.jsdelivr.net/npm/@vladmandic/face-api/model</code>
Purpose	Hosts TensorFlow.js face recognition model weights
Loading	Lazy-loaded on first attendance attempt; browser-cached
Models	ssdMobilenetv1, faceLandmark68Net, faceRecognitionNet

12. Constraints & Assumptions

12.1 Constraints

1. Face recognition runs **entirely client-side** — no server-side ML inference. Accuracy depends on device hardware and lighting conditions.
2. GPS geofencing requires the user's browser to grant location permission. Denied permission blocks GPS-mode attendance.
3. Profile photos are stored as base64 strings in Firestore — Firestore has a 1MB document size limit. High-resolution webcam snapshots are resized before storage.
4. The application requires a modern browser (Chrome 90+, Firefox 88+, Safari 14+) for WebRTC camera access and TensorFlow.js support.
5. All date/time logic uses IST (`Asia/Kolkata`) via `Intl.DateTimeFormat` — system time zones on the device do not affect date calculations.

12.2 Assumptions

1. The admin is a trusted user with full access; no further sub-admin roles are currently defined.
 2. A single office location (Ambience Mall, Gurugram) is geofenced. Multi-location support would require schema changes.
 3. Meeting data is entered manually by the admin, not auto-populated from calendar integrations.
 4. The `appSettings` document ID `main` is singular — the system supports one admin configuration.
-

13. Glossary

Term	Definition
Admin Portal	The management-facing interface at <code>/admin/*</code> , accessible only to the admin user
Employee Portal	The employee-facing interface at <code>/employee/*</code> , scoped to the logged-in employee's data
Face Descriptor	A 128-dimensional float array representing a person's facial geometry, extracted by faceRecognitionNet
Euclidean Distance	The straight-line distance between two face descriptors in 128D space; used as similarity metric
Geofence	A virtual geographic boundary (700m radius circle) centred on the office location
Haversine	A formula for calculating the great-circle distance between two GPS coordinates on a sphere
IST	Indian Standard Time — UTC+5:30 — used for all date/time calculations in the system
MOM	Minutes of Meeting — a structured summary document produced after meetings
onSnapshot	Firebase Firestore real-time listener that fires on every document/collection change
SPA	Single Page Application — the entire app is served as one HTML file; routing is client-side
SRS	Software Requirements Specification — this document
UID	Firebase Auth User ID — a unique string identifier assigned to each authenticated user